

Forge

Beyond grind with structural search and exact certificates

A concrete architecture for automatic induction, lemma discovery, quantifier instantiation, witness synthesis, and nonlinear reasoning

Prepared for Vladimir Reshetnikov

September 14, 2026

Abstract

The most useful way to build a tactic substantially more capable than `grind` is not to replace its congruence closure or arithmetic engines. It is to let a proof-producing controller change the shape of the problem: invent a useful induction motive, generalize a changing accumulator, synthesize a quantified witness, discover a missing lemma, or request a certificate from a stronger arithmetic procedure. These operations must cooperate with saturation instead of running as an unconnected sequence of tactics.

This article specifies `Forge`, a proposed Lean tactic with a typed obligation graph, demand-driven structural search, theory-constrained instantiation, and a strict search/check separation. It identifies concrete Lean libraries and integration points, gives algorithms and mathematical justifications, and includes executed Python prototypes. The experiments produce 215 accepted certificates, replayed without third-party libraries, and 31 passing test methods. They demonstrate mechanisms and expose several failure boundaries; they do not establish a performance advantage over Lean.

Evidence boundary. Lean 4.34.0 and current project documentation were inspected. No Lean executable was available in the execution environment. The Python prototypes and their exact checkers were executed; the proposed end-to-end tactic and the illustrative Lean file were not. All numerical results in this article refer to the supplied Python experiments.

Contents

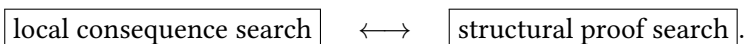
1	Design summary	3
2	The baseline: what should not be reinvented	4
3	A proof-producing obligation graph	5
4	Search control: structural changes before blind saturation	6
5	Automatic induction and demand-driven lemma discovery	7
6	Polynomial recurrence invariants by exact CEGIS	10
7	Synthesize witnesses, not merely satisfying assignments	11
8	Theory-constrained quantifier instantiation	13
9	Nonlinear reasoning with exact certificates	14
10	Lean integration without a second, weaker theorem prover	17
11	Higher-order structure and external reasoning	18
12	Soundness invariants and adversarial failure modes	19
13	A concrete implementation plan	21
14	Executed experiments and their interpretation	22
15	How to demonstrate genuine superiority over grind	25
16	Assessment and recommended starting point	25
A	Artifact inventory and commands	26
B	Certificate representations	27
C	Known limitations of the delivered code	27
	References	28

1 Design summary

Design decision: retain grind as a local reasoning engine and build a structural, certificate-driven proof search system around it. I call this system Forge. The name denotes the proposed design, not an existing released Lean package.

A saturation engine is very effective when useful ground terms and useful lemmas are already present. A much wider class of goals requires making one of those things exist. For example, proving a tail-recursive reverse function correct may require strengthening its specification from an empty accumulator to an arbitrary accumulator. Proving an existential statement may require constructing a function of the universally quantified inputs rather than choosing one concrete value. Proving a quantified consequence may require inventing an arithmetic term that does not occur in a trigger-compatible form.

Forge therefore alternates between two kinds of progress:



It treats grind, simplification, exact arithmetic certificates, lemma applications, and induction as cooperating workers. Every successful worker must return a Lean proof, a replayable certificate with a Lean soundness argument, or explicitly unsolved subgoals. A plausible conjecture is never entered into the proved-fact database.

The main proposed additions are a demand-driven induction and lemma-discovery worker; a synthesis worker for source-level witnesses and arithmetic instantiations; a nonlinear worker with exact certificates; and a controller that preserves proof states, dependencies, and useful facts between these operations. Higher-order and external ATP workers fit the same interface.

This is an engineering synthesis, not a claim that its constituent algorithms are new. Aesop already supplies structured proof search; CCLemma provides important prior art for e-graph-guided inductive lemma discovery; and existing Lean tools provide SMT reconstruction and sum-of-squares reasoning. The value of this proposal is the concrete integration, the demand protocol, and the precise boundary between evidence, conjectures, and checked results. [6, 7, 16, 13]

1.1 What “more powerful” should mean

Here it means *more goals solved automatically from comparable initial information*, particularly goals that need structural proof steps or new terms. It does not mean a stronger logic, a change to Lean’s trusted axioms, or a theorem that no invocation of grind could ever prove the same result. Once the right induction hypotheses and auxiliary lemmas are supplied, a local solver may finish a proof that it would not discover by itself.

Nor can a finite-budget heuristic promise pointwise dominance over another heuristic while taking exactly the same resources. Two operating modes make the tradeoff explicit. A *baseline-preserving* mode runs an ordinary grind attempt with budget B and permits additional search afterward; it preserves that attempt’s successes at additional cost. A *competitive* mode shares one budget among workers; it must be benchmarked and may lose individual baseline successes. These must not be conflated in evaluation.

1.2 What was actually built

The archive contains a typed first-order list prover with structural-induction replay and accumulator generalization; a counterexample-guided polynomial recurrence solver; an affine-witness synthesizer with Farkas certificates; a finite-dictionary square-certificate solver; and an exact Bernstein subdivision solver. The latter two live in one module. Their checkers are executable Python, not formally verified Lean implementations.

The important result is qualitative as well as numerical. The list prover’s fixed-accumulator induction attempt fails, whereas a generated generalization succeeds. The affine-witness worker solves a universally

quantified strip problem for which its constant-witness template fails. Subdivision certifies 16 of the 41 positive-polynomial cases that the root-only Bernstein check does not certify. Those are ablations within the prototypes, not measurements against grind.

2 The baseline: what should not be reinvented

The documented grind architecture already combines congruence closure, propositional propagation, E-matching, guided case splitting, and specialized arithmetic/algebraic reasoning. It produces ordinary Lean proof terms and uses a shared state to let the components exchange consequences. Describing a new e-graph with a linear arithmetic solver as a major improvement would therefore miss the baseline.[1]

The baseline must also be dated. Lean 4.34.0 was released on September 14, 2026. Its `grind => bv_decide` integration encodes relevant equivalence classes and facts learned by theory solvers and E-matching into the SAT problem. Basic cooperation between bit-blasting and grind is consequently *existing functionality*, not a proposed Forge innovation.[2]

Likewise, the linear arithmetic component handles several integer constraint forms and already has model-based theory combination. That is not the same thing as general model-based quantified instantiation, but it rules out a claim that merely exchanging model-induced equalities is new.[3] The release notes also describe grind discharging side conditions for conditional `Sym.simp` theorems. Forge’s proposal is broader cross-worker obligation scheduling, not the invention of guarded rewriting. [2]

2.1 A dependency map, with integration decisions

Component	Reuse	Integration decision
Lean core / grind	Shared equality and local-theory reasoning.	Keep as the default local closer; initially use the tactic boundary.[1, 5]
Aesop	Best-first AND-OR proof search and indexed rules.	Reuse rule-search ideas and adapters rather than claiming a new generic search tree.[6]
Mathlib tactics	<code>simp</code> , <code>ring</code> , <code>linarith</code> , <code>nlinarith</code> , <code>positivity</code> , <code>linear_combination</code> .	Cheap proof reconstruction and preprocessing before expensive searches.[10, 11, 12]
Lean sos	Rational nonlinear certificates and a proof-producing tactic.	Optional production nonlinear backend; do not replace it with the prototype’s restricted LP dictionary.[13]
Duper	Proof-producing automated reasoning inside Lean.	A separate lemma/first-order search worker, not an unchecked external verdict.[14]
lean-auto	Translation and monomorphization infrastructure.	Reuse rather than invent an unverified dependent-to-first-order erasure.[15]
lean-smt / cvc5	SMT search with Lean reconstruction; unsupported proof steps can remain goals.	Optional backend with an explicit residual-obligation interface.[16]
Z3	Candidate models and quantified search techniques.	Untrusted search assistance; a verdict is not a Lean proof.[17]
egg / CCLemma	Equality-saturation and inductive-lemma-search prior art.	Algorithmic references and possible isolated search processes.[8, 7]

The historical lean-egg project is not selected as a new foundational dependency: its README says it is no longer developed and recommends grind.[9]

These projects do not automatically form a build-compatible package set on the day a new Lean release appears. Pin the core toolchain and select mutually compatible Mathlib and optional-backend revisions; keep their adapters separate. The archive does not invent a tested `lake-manifest.json` for a combination that was not built.

3 A proof-producing obligation graph

Implementation status. Specified architecture. The Python prototypes implement several workers and certificate formats, not this Lean controller.

3.1 Three distinct data structures

A single global collection of “facts” is too ambiguous. The controller maintains three stores with different admission rules.

The *proved-fact store* contains propositions together with Lean proof expressions valid in a particular local context. Each fact records its source, its assumptions, and the proof dependencies needed to reconstruct it. A worker may consume it as evidence.

The *obligation graph* contains goals waiting for proofs: ordinary goals, side conditions, candidate lemmas, induction cases, witness specifications, and proof-reconstruction holes. An AND node is solved only after all required children are solved. An OR node records alternative approaches to the same goal. Failed attempts and exhausted budgets do not become logical facts.

The *search store* contains models, candidate terms, speculative rewrites, usefulness scores, and unsuccessful attempts. These may direct search. They are not accepted as hypotheses by another proof worker. In particular, a conjectural equality may merge nodes in an isolated exploration graph, but must not merge nodes in the proved equality state.

3.2 An explicit request protocol

The following is a proposed interface, not a transcription of a released Lean API. The goal identifier denotes an obligation in a snapshot with a known typed context.

```
Demand :=
  Prove(goal)
  ProveGuard(context, proposition)
  FindWitness(context, binders, specification, grammar)
  Instantiate(context, theorem, desired_consequence)
  Generalize(context, recursive_call, blocked_goal)
  FindLemma(context, left_frontier, right_frontier)
  ProveInduction(context, motive, recursor_candidates)
  Reconstruct(context, source_goal, external_certificate)

WorkerResult :=
  Proved(lean_proof, dependencies, replay_record)
  Progress(cheked_facts, new_obligations, continuation)
  Candidate(untrusted_evidence, proposed_next_demands)
  Unknown(reason, diagnostics, continuation)
```

A worker returning `Progress` must not pretend that its residual goals are solved. The controller may tentatively construct a parent proof containing metavariables, but an overall success is reported only after those metavariables are assigned and the completed expression is checked.

A counterexample belongs in `Candidate` unless its terms, hypotheses, and the negation of the source conclusion have all been validated in Lean. For an abstraction-based SMT translation, a satisfying assignment may simply be a model of an overapproximation.

3.3 The snapshot and its identity

A proposed GoalSnapshot contains the environment revision, local context, local instances, metavariable context, target expression, transparency configuration, branch assumptions, normalization version, and worker state. The controller must restore *all* of the relevant state on backtracking. Restoring just the visible list of hypotheses is insufficient: a worker may have instantiated metavariables or created local declarations with identities used by other expressions.

A persistent fact key includes its type and context identity, not just the pretty-printed proposition. Names printed as x can denote different free variables in sibling goals. Implicit typeclass arguments and universe instances also matter. Two syntactically similar applications of an overloaded operation can have different meanings because their instances differ.

Pure syntax nodes can be shared globally when their identities are stable. Proof-bearing facts are shared only when their assumptions can be transported into the destination context. One safe reuse method abstracts a proof over its exact free-variable dependencies and then explicitly instantiates that closed theorem in the destination context.

3.4 Facts carry explanations, not only truth flags

For example, a nonlinear worker may prove $0 < c$ from polynomial constraints. That proof justifies $c \neq 0$, which unlocks a division rewrite. The rewrite’s proof yields an equality, which may enable congruence closure, which may make an E-matching instance useful. The sequence should be represented as a proof DAG with explicit edges, not repeated reparsing of tactic text.

A fact introduced under a split assumption b has a dependency on b . It cannot be used in the $\neg b$ branch. If both branches prove the same proposition, their proofs are combined using the appropriate eliminator. There is no “most likely branch” rule in the logical layer.

4 Search control: structural changes before blind saturation

4.1 The main loop

```
solve(snapshot, budget):
  normalize_and_introduce(snapshot)
  queue = initial_demands(snapshot)
  while queue is nonempty and budget remains:
    demand = fair_priority_pop(queue)
    state = restore(demand.snapshot)
    result = select_worker(demand).run(state, local_budget)
  match result:
    Proved(p, deps, record):
      validate_in_exact_context(p, demand.goal)
      commit_proof(demand, p, deps, record)
      wake_dependent_obligations()
    Progress(facts, children, continuation):
      validate_and_commit_each_fact(facts)
      add_children_without_assuming_them(children)
      schedule(continuation)
    Candidate(evidence, suggestions):
      store_as_untrusted(evidence)
      schedule(suggestions)
    Unknown(reason, diagnostics, continuation):
      record_failure_without_logical_inference(reason)
      schedule_if_budget_policy_allows(continuation)
  return checked_root_proof_or_explicit_failure()
```

A useful first implementation is single-threaded and deterministic. Search parallelism is optional, and is not a prerequisite for stronger reasoning. External processes can propose certificates independently, but a single Lean context owns proof reconstruction and commits.

4.2 Selecting the next action

The controller detects the shape of the obstruction rather than increasing all budgets indiscriminately. A recursive call whose accumulator changes suggests generalization. A goal beginning with an existential suggests witness synthesis. A universal theorem with an arithmetic-shaped trigger suggests constrained instantiation. A nonnegative polynomial goal with usable bounds suggests an arithmetic certificate. An equality of functions suggests an extensionality step before first-order reasoning.

The priority score can initially be a transparent hand-tuned combination of estimated cost, overlap with target symbols, number of blocked dependents, recursion relevance, and past unsuccessful attempts. These are engineering heuristics, not calibrated probabilities. Keep a fair queue lane with growing budgets so that a continuously regenerated cheap task cannot starve every structural attempt. Every global budget must also account for reification, proof reconstruction, and kernel checking.

4.3 Do not retrieve the entire library

Maintain indexes by conclusion head, referenced recursive definitions, argument types, equality orientation, and the heads of premises. A blocked obligation drives the query. Start with local hypotheses, equations of the mentioned definitions, relevant extensionality lemmas, and a small retrieved lemma set. Expand retrieval only if these fail.

A discrimination-tree or comparable structural index provides the first filter; a dependency-aware score ranks the survivors. Candidate lemma applications are elaborated in the exact goal context before they can affect search. Text similarity or an optional language model can propose candidates, but the system does not need either for the algorithms demonstrated here.

Aesop's best-first rule organization is a practical model for the outer search. Forge adds domain-specific demands and cross-worker continuations rather than claiming that AND-OR search itself is novel.[6]

4.4 A representative cooperative proof

Consider a program theorem that combines a tail-recursive list function with a polynomial cost recurrence. The controller can discover an accumulator specification, prove it by list induction, synthesize a closed form for the cost by arithmetic induction, and pass the resulting equalities to grind to close the original verification condition. Neither generated theorem is asserted until its own proof is complete.

The important distinction from `first | grind | aesop | nlinarith` is that unsuccessful attempts still produce structured, reusable information: which recursive argument changed, which equality would enable progress, which witness template was insufficient, and which arithmetic side condition remains. This feedback directs the next attempt, and checked intermediate results return to the shared proof state.

5 Automatic induction and demand-driven lemma discovery

Implementation status. The list fragment, changing-argument detection, bounded RHS synthesis, structural induction, and trace replay are implemented. General dependent induction and e-graph-guided frontier discovery are specified but not implemented.

5.1 Choose an induction variable from recursive structure

Collect recursive applications in the definitions mentioned by the goal and its relevant hypotheses. For each function, record which argument positions are destructured by pattern matching, which are passed to recursive calls as proper subterms, and which non-structural arguments change. Rank induction

candidates by how many blocked recursive calls they expose after one constructor split. Do not select a variable merely because its type has an induction principle.

For a first implementation, restrict automatic induction to standard structural recursion over supported inductive types. A later adapter can use functional or well-founded induction principles associated with a recursive definition. It must construct an actual Lean recursor application; recognizing a decreasing-looking expression is not a replacement for a termination or well-foundedness proof.

The candidate motive is a typed function from the induction index to a proposition. Generalize parameters whose values vary in recursive calls. Close that set under dependencies in the local telescope: if a generalized variable occurs in the type of another variable or hypothesis, that dependent binder must be generalized or explicitly transported as well.

For instance, in a context containing $x : A$, $y : B(x)$, and $h : C(x, y)$, induction on x may require a motive of the form

$$M(x) := \forall y : B(x), C(x, y) \rightarrow P(x, y),$$

not a proposition containing the old fixed y at every new value of x . In Lean, use elaboration and the existing induction machinery to build this motive; do not manufacture a well-typed-looking first-order erasure.

5.2 The accumulator example, including the obstruction

Write A for append, R for ordinary reverse, and RA for accumulator reverse. The defining equations are

$$\begin{aligned} A([], y) &= y, & A(h :: t, y) &= h :: A(t, y), \\ R([]) &= [], & R(h :: t) &= A(R(t), [h]), \\ RA([], a) &= a, & RA(h :: t, a) &= RA(t, h :: a). \end{aligned}$$

Suppose the requested theorem is

$$RA(x, []) = R(x).$$

Inducting on x without generalization gives an induction hypothesis only about $RA(t, [])$. But the recursive call in the step case is $RA(t, [h])$. The induction hypothesis does not match. Increasing a rewriting budget cannot make this particular hypothesis say something stronger.

The changing-argument detector sees that argument 0 is destructured structurally and argument 1 changes from a to $h :: a$. It replaces the fixed empty accumulator with a fresh parameter and asks for a right-hand side in a typed grammar:

$$L ::= [] \mid x \mid a \mid R(L) \mid A(L, L).$$

The successful candidate is

$$\forall x a, \quad RA(x, a) = A(R(x), a). \tag{1}$$

The base case is immediate. For the step case, the generalized induction hypothesis gives

$$RA(t, h :: a) = A(R(t), h :: a).$$

The target's other side is

$$A(R(h :: t), a) = A(A(R(t), [h]), a) = A(R(t), A([h], a)) = A(R(t), h :: a),$$

where the middle equality uses append associativity. Instantiating (1) at $a = []$ and using append's right identity proves the original theorem.

The implemented grammar enumerator finds this RHS as candidate 32. It filters candidates using all 49 pairs of lists of length at most two over the alphabet $\{0, 1\}$. Exactly one candidate reaches the proof search before success. These finite examples are *not* its proof: the candidate then passes symbolic induction and independent trace replay.

5.3 What the miniature checker actually permits

There are separate syntax constructors for universal pattern variables and rigid free variables. In an induction step, the tail is a rigid free variable. Other generalized parameters remain variables of the induction-hypothesis rule. Thus the hypothesis can be instantiated at a new accumulator, but cannot be applied to an arbitrary longer list.

Every rewrite certificate contains a rule identifier and a subterm path. The checker retrieves the rule from the defining equations, an earlier checked lemma, or the current local induction hypothesis. It matches the left-hand side, checks repeated-variable identity and sorts, replaces the selected subterm, and continues. The endpoints of the two replayed sides must be syntactically identical.

The public theorem-installation operation first checks both induction cases. A lemma is not available while its own certificate is being checked. This rejects a forged proof that cites its own conclusion. The prototype does not implement simultaneous induction; a production extension would need an explicit joint motive and recursor rather than relaxing this restriction.

Proposition 5.1 (Soundness of the list certificate format). *Assume that previously installed equations are valid for all finite lists. If the miniature checker accepts an induction certificate, its universally quantified equation holds for all finite lists under the supplied definitions.*

Proof. Each defining rewrite preserves the interpreted value. Each installed lemma instance preserves it by the assumption, and congruence permits replacement inside a typed term context. Consequently an accepted base trace proves the empty-list case. In the step trace, the only additional rule is the desired statement for the rigid tail, universally quantified over its generalized parameters. That rule is valid under the structural induction hypothesis. The accepted step trace therefore proves the constructor case. Structural induction establishes the universal equation. $\square$

This is a mathematical justification of a small format, not a machine-checked Lean proof of the Python implementation. Bugs in the checker remain possible; the negative tests and replay separation reduce that risk but do not turn Python into Lean’s kernel.

5.4 Discover the helper lemma, not only the generalization

The prototype supplies four seed *statements*: append associativity, append’s right identity, reverse of append, and reverse involution. It searches and checks their proofs, but does not claim to have invented all four statements. In particular, associativity is available when the accumulator lemma is synthesized.

The full design adds a demand-driven lemma explorer. After a failed induction step, retain the normalized left and right frontiers and the rewrites that led to them. Bounded equality saturation can expose alternative forms that a one-way normalizer would discard. For the accumulator example, unfolding append in reverse can expose $h :: a$ as $A([h], a)$. The remaining frontier then suggests

$$A(A(u, v), w) = A(u, A(v, w)).$$

Prove that generalized statement separately, and only then install it.

A practical candidate-generation algorithm is:

```
find_lemma(left_frontier, right_frontier):
    expand each frontier using a bounded set of proved equalities
    identify pairs separated by a small structural difference
    anti-unify shared subterms, preserving repeated-variable identity
    abstract a well-typed parameter telescope
    reject trivial, cyclic, or irrelevant candidates
    filter candidates with executable counterexamples where available
    prove surviving candidates in a separate obligation
    return only checked lemmas to the main equality state
```

Typed anti-unification starts in a deliberately restricted fragment: fully applied, homogeneous first-order terms with compatible types. Memoize pairs of subterms so that anti-unifying $f(a, a)$ and $f(b, b)$ gives $f(z, z)$, not $f(z_1, z_2)$. In a dependent fragment, any abstraction that changes a term's type needs a typed telescope and possibly transport proofs. Failure to find such an abstraction is an ordinary search failure.

This goal-directed exploration is strongly related to CCLemma's approach. Its authors use e-graphs to focus inductive lemma discovery on useful proof frontiers. The present proposal adapts that line of work to a Lean proof-producing controller; it does not claim to originate the method. [7]

5.5 Budgeting and termination of exploration

Impose separate limits on term size, saturation nodes, candidate count, induction depth, and lemma-dependency depth. Use term fingerprints and alpha-normalized candidate telescopes to avoid reproving the same lemma under new binder names. Never accept a cyclic dependency merely because every edge looks like a useful lemma application.

Bounded enumeration is exponential in its grammar size. The initial list experiment is intentionally tiny. The production advantage must come from recursive-call analysis, relevance filtering, typed indexing, and frontier selection, not from assuming unrestricted enumeration will be cheap.

6 Polynomial recurrence invariants by exact CEGIS

Implementation status. Implemented and tested on 65 recurrence problems, including five power-sum families and 60 seeded random polynomial increments.

6.1 The fragment and the certificate

Consider the recurrence

$$T(0) = t_0, \quad T(n+1) = T(n) + f(n), \quad n \in \mathbb{N}, \quad (2)$$

where $t_0 \in \mathbb{Q}$ and $f \in \mathbb{Q}[X]$. Search for $q \in \mathbb{Q}[X]$ such that

$$q(0) = t_0, \quad q(X+1) - q(X) = f(X). \quad (3)$$

The certificate is just the exact coefficient list of q . Its checker computes the shift $q(X+1)$ using polynomial arithmetic and compares exact coefficients. It does not sample the step relation or trust an interpolator.

Proposition 6.1. *If (3) holds, then $T(n) = q(n)$ for every $n \in \mathbb{N}$.*

Proof. The base equality is the first certificate condition. If $T(n) = q(n)$, then (2) and the second condition imply $T(n+1) = q(n) + f(n) = q(n+1)$. Apply induction on n . $\square$

In Lean, this certificate can be reconstructed with ordinary induction, explicit casts, and polynomial normalization. A compact reflected checker is an optimization, not a requirement for the first implementation.

6.2 Counterexamples direct the coefficient search

For degree bound d , write $q(X) = \sum_{i=0}^d c_i X^i$. Maintain exact sample constraints $q(n) = T(n)$ and solve the resulting rational linear system. Free coefficients are initially set to zero. Check the candidate's base and step identities. If the step residual

$$r(X) = q(X+1) - q(X) - f(X)$$

is nonzero, choose a natural n with $r(n) \neq 0$ and add both $q(n) = T(n)$ and $q(n+1) = T(n+1)$ to the sample set. At least one is a new constraint: if both were already satisfied by the candidate, their difference would force $r(n) = 0$.

A nonzero degree- k polynomial cannot vanish at all of $0, 1, \dots, k$. The implementation uses this elementary fact to find an exact counterexample without an SMT solver. If the coefficient constraints become inconsistent, increase the degree bound. The proposal is therefore counterexample-guided, while its acceptance test is an exact induction certificate.

6.3 A completeness statement for this small fragment

For a nonzero polynomial f of degree k , the finite-difference map $q \mapsto q(X+1) - q(X)$ sends a leading term aX^{k+1} to $(k+1)aX^k$ plus lower terms. Over $\mathbb{Q}$, solve for the leading coefficient, subtract its difference, and continue downward. Thus there is a degree $k+1$ solution, unique after fixing $q(0)$. The zero increment gives a constant solution.

For a fixed candidate degree, each unsuccessful iteration adds an independent constraint or detects inconsistency. Hence exact linear solving and a degree budget at least $k+1$ suffice for termination on this fragment. This does not assert completeness for arbitrary recurrences, nonlinear state updates, piecewise definitions, or integer formulas involving truncated division.

6.4 An example and an overfitting trap

For $f(X) = X + 1$, the discovered result is

$$q(X) = \frac{X(X+1)}{2}.$$

For increments $(X+1)^2, \dots, (X+1)^5$, the output contains the corresponding power-sum polynomials; the exact formulas and all failed candidate residuals are in `results/certificates/recurrences.json`.

An interpolated expression

$$\tilde{q}(X) = \frac{X(X+1)}{2} + X(X-1)(X-2)(X-3)$$

agrees with the triangular-number formula on the first four inputs. It is nevertheless rejected by the step checker. This is a small, concrete example of why testing-based conjecture generation and proof acceptance must be separate operations.

6.5 Generalization to richer invariants

A next worker can search polynomial invariants $I(n, s)$ for a state transition $s' = F(n, s)$. It must prove the initial condition and the implication

$$I(n, s) \wedge \text{Guard}(n, s) \implies I(n+1, F(n, s)).$$

Coefficient comparison handles identities; exact nonlinear certificates can handle some inequalities. The invariant template and degree bound are explicit search parameters. Counterexamples refine coefficients or force a richer motive. This is a concrete extension path, but the supplied recurrence module implements only the additive univariate fragment above.

7 Synthesize witnesses, not merely satisfying assignments

Implementation status. Affine witness synthesis with exact Farkas certificates is implemented. Piecewise witnesses, integer witnesses, and dependent-type witness synthesis are proposed extensions.

7.1 The logical distinction

To prove $\forall x \exists y P(x, y)$, a satisfying assignment for one value of x is not enough. A useful synthesizer produces a term $y = w(x)$ and proves $\forall x P(x, w(x))$. For example,

$$\forall x \in \mathbb{R}, \quad \exists y \in \mathbb{R}, \quad x + 1 \leq y \leq x + 2$$

has the witness $w(x) = x + 1$. No constant witness works for every real x . The prototype’s constant-only template returns no certificate; its affine template returns the source-level witness $x + 1$ with an exact check.

In the Lean controller, witness binders retain their dependencies. A witness for the second existential may refer to an earlier witness, but not to a future binder or a branch-local variable outside its scope. A witness is elaborated as an ordinary Lean term before its specification is discharged.

7.2 Jointly solve for an affine witness and its proof

Let the input domain be the rational polyhedron

$$g(x) = Ax + b \geq 0,$$

and the target constraints be

$$h(x, y) = Cx + Dy + e \geq 0.$$

All inequalities are componentwise and non-strict. Seek $y = Mx + c$. A sufficient certificate consists of a nonnegative matrix Λ and nonnegative vector s satisfying

$$C + DM = \Lambda A, \tag{4}$$

$$e + Dc = \Lambda b + s. \tag{5}$$

These equations are *linear in the unknowns* M, c, Λ, s , because A, b, C, D, e are fixed problem data. Witness synthesis and proof search can therefore be one linear feasibility problem, rather than enumerating many candidate affine functions and invoking a verifier separately.

Proposition 7.1 (Affine witness certificate). *If (4)–(5) hold and $\Lambda, s \geq 0$, then for every x satisfying $Ax + b \geq 0$, the term $y = Mx + c$ satisfies $Cx + Dy + e \geq 0$.*

Proof. Substitution gives $Cx + D(Mx + c) + e = \Lambda(Ax + b) + s$. Every entry on the right is a sum of nonnegative multiples of nonnegative quantities, plus a nonnegative slack. $\square$

For a nonempty rational polyhedron, the affine implication form of Farkas’ lemma also gives a completeness route for a *fixed affine witness*: validity of each non-strict affine target inequality has such multipliers. The combined exact feasibility formulation therefore captures the existence of affine witnesses in that fragment. This is the standard polyhedral dual-certificate reasoning.[21] The floating-point proposer is not a complete exact LP algorithm, and an empty domain needs separate handling for a completeness claim. Neither qualification affects certificate soundness.

7.3 Implementation details and exact reconstruction

With n inputs, m outputs, r domain constraints, and q target constraints, the encoding uses

$$m(n + 1) + q(r + 1)$$

unknowns and $q(n + 1)$ coefficient equations, before template restrictions. Entries of M, c are unrestricted; entries of Λ, s are constrained nonnegative. A constant-only grammar adds equations $M = 0$.

The prototype calls SciPy’s `linprog` with the HiGHS backend. That API is an optimization interface, not a proof system.[20] The returned numbers are rationalized and all coefficient equations and sign constraints

are checked with exact fractions. If that fails, a second attempt uses the numerically suggested support and exact rational elimination. If the exact check still fails, the answer is “no certificate found.”

A numerical zero threshold is used only to select a support for the second search attempt. There is no numerical tolerance in the acceptance condition. The serialized certificate includes each witness, each multiplier row, and each slack. Its replay code imports no optimization library.

For Lean reconstruction, produce the witness tuple first, then prove every target inequality from the recorded nonnegative combination. The coefficient identity can be checked by polynomial normalization, and the sign proof can be built from the domain hypotheses. One can use ordinary tactic reconstruction initially and a reflected matrix checker later.

7.4 Beyond a single affine template

There are valid specifications with no witness in the chosen grammar. For piecewise-affine witnesses, permit a bounded decision tree whose guards are source-level affine comparisons. Synthesize an affine witness under each leaf’s domain assumptions, and prove the guard partition covers the original domain. Never silently assume that finitely many sampled regions cover it.

For integer or natural-number witnesses, a rational affine solution is not sufficient. Require integral coefficients where that suffices, or introduce explicit division/remainder templates with associated arithmetic obligations. A practical first integer grammar is

$$w(x) ::= c + \sum_i a_i x_i \mid \left\lfloor \frac{c + \sum_i a_i x_i}{d} \right\rfloor \mid \text{ite}(g, w_1, w_2), \quad d > 0.$$

Each candidate must be checked in the source integer semantics. The current prototype deliberately does not interpret a real solution as an integer proof.

8 Theory-constrained quantifier instantiation

Implementation status. Specified algorithm; no general E-matching or MBQI implementation is included in the prototype archive.

8.1 Solve for missing terms before increasing instance limits

Trigger-based instantiation works from terms present in the equality state. The documented E-matching machinery is therefore an important baseline, not something to discard.[4] *Forge* adds a demand that works backward from a desired consequence to source terms for the theorem’s binders.

Consider an integer predicate P , hypotheses

$$\forall k \in \mathbb{Z}, P(2k + 1), \quad n \bmod 2 = 1,$$

and goal $P(n)$. A useful instantiation is $k = n / 2$. The controller can synthesize that term from the arithmetic shape, then prove

$$2(n / 2) + 1 = n$$

using the oddness assumption and integer division facts. It applies the original universal theorem and transports along this checked equality.

This is an algorithmic example, not a measured claim that the current grind fails on that exact input. The included Lean example shows the intended proof shape but was not compiled.

8.2 A concrete constrained-matching algorithm

Given a selected theorem conclusion $C(z)$ and a requested atom G , first perform typed syntactic matching on the uninterpreted structure. Instead of failing at an interpreted arithmetic subterm, emit an equality constraint between that subterm and the corresponding source expression.

For linear integer patterns, solve the resulting constraints for missing binders using divisibility, linear elimination, or a bounded grammar of quotient/remainder terms. For affine rational patterns, exact linear solving can directly produce terms. For constructor patterns, use constructor injectivity and source-level constructor terms. Underconstrained binders are handled by a bounded typed grammar, not arbitrary model objects.

Each candidate substitution θ creates ordinary proof obligations for the matching equalities and any theorem premises. After those are proved, construct the theorem application $C(\theta)$ and its transport to G . Nothing about the search solver needs to be trusted: an incorrect candidate simply fails reconstruction.

Several triggers or repeated binders produce a *joint* constraint system. Solving each occurrence independently would lose the equality relationship between two occurrences of the same variable. Cache instances using the theorem identity, universe/type instantiations, the complete substitution, and its valid branch context.

8.3 Model-guided instantiation, with a narrow trust boundary

If constrained matching is insufficient, use a model-based loop inspired by quantified SMT procedures. Candidate-model and pattern-based instantiation are established techniques in systems such as Z3.[17] They can suggest useful instances even when naive trigger firing stagnates.

The proposed loop freezes a relevant typed abstraction, obtains a candidate model, and searches for a violated universal instance in that model. It then tries to reify the candidate values as source terms and verifies the associated instance in Lean. Numerals can often be reified directly. An uninterpreted model-domain element cannot simply become a new arbitrary Lean constant: find a denoting term already available, enumerate a term, or abandon that candidate. Internal Skolem symbols likewise require scoped witnesses or hypothesis elimination, not new axioms.

A satisfying model of an abstraction is not automatically a countermodel of the Lean goal. Conversely, an external unsatisfiability result is not accepted without proof reconstruction or an appropriate verified certificate. In this architecture, model search is an instance generator; Lean remains responsible for every asserted consequence.

Generation limits, trigger selectivity, repeated-instance suppression, and fair round scheduling are still necessary. There is no claim that this loop is complete for arbitrary quantified dependent type theory.

9 Nonlinear reasoning with exact certificates

Implementation status. Finite-dictionary square search and Bernstein subdivision are implemented. A full SDP solver and its Lean integration are reused/proposed, not implemented by these Python modules.

9.1 A nonlinear worker should expose multiple certificate classes

Start with cheap normalization and obvious sign facts. Use Mathlib's `positivity` and arithmetic tactics as proof-producing preprocessors or closers where appropriate.[11, 10] Then select a more specialized certificate class according to the target and assumptions: algebraic identities, nonnegative square combinations, bounded-box Bernstein certificates, or an optional stronger semialgebraic backend.

Do not confuse polynomial equality reasoning with order reasoning. Reducing polynomials modulo equalities does not by itself prove nonnegativity. Conversely, knowing a square is nonnegative does not solve all nonlinear integer arithmetic problems.

9.2 Square certificates and the existing Lean SOS backend

For constraints $g_i(x) \geq 0$ and equations $e_j(x) = 0$, a useful certificate shape is

$$p = \sum_k a_k q_k^2 + \sum_{i,k} b_{ik} r_{ik}^2 g_i + \sum_j h_j e_j, \quad a_k, b_{ik} \in \mathbb{Q}_{\geq 0}. \quad (6)$$

Checking the polynomial identity and the rational signs proves $p \geq 0$ under the source constraints. The polynomial multipliers h_j of equalities need no sign condition.

There is already a Lean `sos` project implementing proof-producing nonlinear reasoning with rational certificates. Its documented architecture separates reification, search, checking, and verification; named interfaces include `SOS.Engine.solve`, `SOS.Engine.Result.check`, and the Mathlib-side verifier. `sos?` can emit a replayable witness. Its computational polynomial substrate uses `Hex.MvPoly`. [13] These are concrete integration targets, subject to revision compatibility. The **Forge** contribution is when to request such a certificate and how to return its proved consequences to structural and saturation workers.

9.3 What the implemented square search does

The prototype uses only the first two sums of (6). It chooses a finite dictionary of candidate square roots, constructs their squares and constraint-multiplied squares, and solves for nonnegative rational weights. Once the roots are fixed, coefficient matching is a linear feasibility problem. For two variables and root degree at most two, the default dictionary has 36 roots: monomials and their pairwise sums and differences.

This is simpler and much less general than searching over arbitrary positive semidefinite Gram matrices. It is useful as a cheap stage and as an experiment in exact search/replay separation, not as a proposed replacement for a full SOS implementation. Its failure proves only that this search did not find a certificate. In particular, a nonnegative polynomial need not be a sum of polynomial squares at all.

The checker reconstructs the whole polynomial from the supplied weights and roots, checks each weight exactly, verifies constraint indices, and compares the result with the target supplied by the caller. A weight of -1 cannot be smuggled through because the polynomial identity happens to be correct. Removing a necessary constraint from the problem also makes replay fail.

9.4 Bernstein certificates on rational boxes

Suppose the domain is a rational box $B = \prod_{i=1}^m [a_i, b_i]$, with $a_i < b_i$. Substitute $x_i = a_i + (b_i - a_i)t_i$ to obtain

$$\tilde{p}(t) = \sum_{\beta} c_{\beta} t^{\beta}, \quad 0 \leq t_i \leq 1.$$

Let d_i be a coordinatewise degree bound. Define

$$B_{\alpha,d}(t) = \prod_i \binom{d_i}{\alpha_i} t_i^{\alpha_i} (1 - t_i)^{d_i - \alpha_i}.$$

Each basis function is nonnegative on the unit box and their sum is one. The Bernstein coefficients are

$$b_{\alpha} = \sum_{\beta \leq \alpha} c_{\beta} \prod_i \frac{\binom{\alpha_i}{\beta_i}}{\binom{d_i}{\beta_i}}. \quad (7)$$

Thus $p \geq 0$ on B if every $b_{\alpha} \geq 0$; it is strictly positive if every $b_{\alpha} > 0$.

Proposition 9.1 (Bernstein coefficient identity). *The coefficients in (7) satisfy $\tilde{p}(t) = \sum_{\alpha} b_{\alpha} B_{\alpha,d}(t)$.*

Proof. In one variable, use

$$t^k = \sum_{j=k}^d \frac{\binom{j}{k}}{\binom{d}{k}} \binom{d}{j} t^j (1-t)^{d-j}.$$

The binomial identity $\binom{d}{j} \binom{j}{k} / \binom{d}{k} = \binom{d-k}{j-k}$ reduces the right-hand side to $t^k(t + (1-t))^{d-k} = t^k$. Take products across coordinates and apply linearity to the monomial expansion of $\tilde{p}$. $\square$

Negative Bernstein coefficients are *not* counterexamples. They mean only that this sufficient condition has not proved the desired sign. A search worker can split a box at a rational midpoint and repeat the check. A returned negative point is a different result and must be evaluated exactly.

9.5 The partition certificate is part of the proof

The implemented certificate is a binary tree. A leaf asks the checker to recompute the Bernstein coefficients for its inherited box. An internal node specifies an axis, a rational split point strictly inside that axis's interval, and both children. The child boxes are derived by the checker, not accepted as arbitrary boxes from the certificate.

This design makes coverage explicit. There is no way to omit a difficult corner by supplying an incomplete list of favorable boxes. A missing child, an invalid split axis, or a split point at an interval endpoint is rejected. The root box and polynomial are separate problem inputs.

For

$$p(x) = \left(x - \frac{1}{2}\right)^2 + \frac{1}{100}, \quad x \in [0, 1],$$

the degree-two coefficients at the root are $13/50, -6/25, 13/50$. The root check cannot certify positivity. Splitting at $1/2$ gives coefficient lists

$$(13/50, 1/100, 1/100), \quad (1/100, 1/100, 13/50),$$

so two leaves suffice for strict positivity. This is an exact calculation, not a numerical plot or an assertion based on sampled values.

9.6 Limits, completeness, and complementary strengths

For a polynomial strictly positive on a compact box, sufficiently fine subdivision eventually makes every Bernstein coefficient positive. A direct justification is that the transformed higher-degree coefficients tend uniformly to zero as box widths shrink, while the constant terms stay above a positive minimum. Consequently every local Bernstein coefficient is $p(a) + O(\max_i(b_i - a_i))$ with a uniform bound over the original box. The prototype cycles split axes so every coordinate shrinks along a branch. An unlimited fair subdivision search therefore has a useful completeness property for strictly positive polynomials on a fixed compact box.

Nonnegative polynomials with zeros are different. The prototype returns unknown for $(x - y)^2$ on $[0, 1]^2$ at depth four, even though a single square certificate proves the result immediately. A search timeout or an uncertified leaf must not be reported as a refutation.

Coordinatewise degrees create $\prod_i (d_i + 1)$ Bernstein coefficients, and a binary tree can have exponentially many leaves in its depth bound. Exact coefficient bit sizes also grow under repeated substitution. This procedure is attractive for low-dimensional bounded problems, not a universal substitute for nonlinear arithmetic.

9.7 Integrating domains, casts, and division safely

The controller must establish that the source assumptions imply the box bounds before applying a Bernstein theorem. Polynomial reification must preserve variable identity, source operations, and casts. A real-field certificate cannot be applied directly to an arbitrary ring or a finite characteristic field.

For natural or integer inequalities, explicitly prove the embedding into an ordered field and translate the relevant operations. Integer division, truncated subtraction, and modular arithmetic are not field operations. Clearing denominators in inequalities requires a sign proof; clearing a field denominator in an equality generally requires a nonzero proof. These become ProveGuard obligations. If the guard fails, the rewrite is unavailable, not approximately valid.

A checked sign or equality from this worker is useful beyond its original goal. It can unlock a conditional rewrite, expose a useful quantified instance, or finish an induction step. That is why the arithmetic worker returns individual proved consequences and their assumptions, not just a Boolean “solved” flag.

10 Lean integration without a second, weaker theorem prover

Implementation status. Integration specification. The cited core names were inspected; the adapter interfaces below are proposed and have not been compiled.

10.1 Begin at a stable boundary, then make selected paths incremental

The first Lean implementation should invoke grind on ordinary Lean goals and supply newly proved local facts explicitly. This is less efficient than an ideal native integration, but it permits the structural planner and certificate pipeline to be tested without depending immediately on every internal representation of grind.

There are concrete lower-level entry points to investigate next. In the inspected Lean 4.34.0 source, `Lean.Meta.Grind.Params` includes `extraFacts`, and `assertExtra` infers the types of those proof expressions and adds them to the goal’s fact machinery. The source also exposes `GrindM.run`, `mkGoalCore`, and a `Result` containing `failure?`, `diagnostics`, and `counters`.^[5] These are implementation hooks, not a promise of a stable public integration contract.

In particular, feeding `Params.extraFacts` into a fresh run is not the same as resuming an earlier saturation state. A true incremental adapter must preserve the goal state and all relevant solver/context state, and must use the normal internalization and propagation path when adding facts. Otherwise an equality might exist in one table while being invisible to another solver.

A proposed versioned adapter has the following surface:

```
GrindAdapter.open(snapshot, options) -> session
GrindAdapter.seed(session, checked_proofs) -> session
GrindAdapter.advance(session, work_quota) -> local_result
GrindAdapter.exportRelevant(session, demand) -> checked_summary
GrindAdapter.snapshot(session) -> resumable_state
GrindAdapter.restore(resumable_state) -> session
```

Keep all version-specific internals in one module. Unit-test seeded facts, case-split rollback, proof extraction, metavariable changes, and equality propagation through every enabled local theory. Until that adapter is reliable, restarting a local tactic with explicit checked facts is preferable to a fast but incoherent shared state.

10.2 Use a typed semantic boundary for each external worker

A reifier does more than turn Lean expressions into strings. For a polynomial fragment, it produces a variable table, an exact polynomial, and a proof that the polynomial’s denotation under that table is the source expression. The source type, operations, and their instances are part of the reification. Unsupported subexpressions may become opaque atoms when that abstraction is sound for the requested procedure.

A schematic contract is

$$\text{reify}(e) = (\text{environment } \rho, \text{IR } t, \pi : e = \text{eval}_\rho(t)).$$

For an inequality or formula, the contract is an equivalence or a suitable one-way implication between the source and reified statements. The direction must match how the solver result will be used.

If congruence closure identifies two source terms, a worker can share one polynomial variable only after preserving the equality proof needed to justify that identification. It must not merge different variables because their printed names happen to agree. Conversely, treating equal terms as different opaque variables is usually a loss of proving power rather than an unsound strengthening; this explains why the shared-state integration matters.

For dependent expressions, equality classes must be homogeneous up to justified type compatibility. Heterogeneous equality is not an excuse to ignore type indices. Keep HEq reasoning and transport explicit, and use Lean elaboration to validate casts. An initial external term language should remain first-order and homogeneous instead of implementing an ad hoc untyped approximation to all of Lean.

10.3 Three levels of proof reconstruction

The simplest backend returns ordinary Lean theorem applications and tactic steps. This is appropriate for synthesized witnesses, explicit induction motives, small nonnegative linear combinations, and recurrence identities. The search can be complicated while replay remains short.

For larger algebraic results, return a typed certificate to a reflected Lean checker with a theorem of the form

$$\text{check}(P, C) = \text{true} \implies \text{Denotes}(P).$$

The problem P comes from the source reifier. The external solver is allowed to choose only C , not silently replace P . In a conservative configuration, reduce the checker inside the trusted proof-checking path rather than accept an unchecked FFI result.

For general external ATP/SMT proofs, use a proof importer with explicit supported rules. `lean-smt` is a concrete candidate: its documented behavior includes returning proof-reconstruction holes as goals, and its supported theories are not equivalent to the entire capability set of `cvc5`. [16] Those holes enter the obligation graph. An external unsat response does not bypass them.

10.4 Keep the efficient existing bit-vector path

A mixed goal involving opaque functions and bit-vectors should use the existing shared-state `grind => bv_decide` route when applicable. Forge can schedule that route after structural steps or new lemmas have made relevant facts available, but should not duplicate its encoding just to fit a uniform architecture.[2]

A modular integer-to-bit-vector connection needs the correct modular semantics. An integer embedding into a field can be injective; a fixed-width bit-vector projection is not. Mixing those relationships in a generic “numeric cast” cache would be a serious error. Reifiers and proof bridges must be indexed by the actual source and target operations.

11 Higher-order structure and external reasoning

11.1 A focused source-term worker

Before translating a goal to a first-order backend, expose its constructive structure. For a function equality, attempt the relevant extensionality principle and introduce a fresh argument. For a universally quantified goal, introduce its binder. For a conjunction or constructor target, create the corresponding subgoals. For an existential, construct a scoped witness and then prove its specification.

A bounded type-directed term synthesizer can handle witness goals beyond the affine fragment. Its grammar includes local terms, lambda abstraction, application of retrieved lemmas/functions, constructors, and selected recursor/fold schemas. Each candidate is elaborated before semantic checking. Higher-order pattern unification handles tractable application patterns; unrestricted higher-order unification is not assumed to be a cheap or complete subroutine.

For example, to synthesize a function $h : \text{List}(A) \rightarrow \text{List}(B)$ with the equations

$$h([]) = [], \quad h(x :: xs) = f(x) :: h(xs),$$

a list-fold schema already fixes the recursion structure. Search only needs the empty case and the constructor step. A library index may retrieve `map`; otherwise a recursor term provides an equivalent witness whose equations are proved by reduction. This is a specific proposed worker, not something the Python first-order list module already implements.

The architecture leaves room for more advanced type-inhabitation search. Canonical is relevant prior work on proof synthesis involving higher-order and dependent types.[18] Such a worker should be integrated by its proof-term interface, not by flattening away the dependencies that make its synthesis task meaningful.

11.2 First-order ATP workers are complementary

After structural exposure, a saturation-based ATP may find useful theorem combinations that a local tactic misses. `Duper` is an in-Lean proof-producing option.[14] An external solver can also be used through a deliberate translation and reconstruction layer. `lean-auto` provides relevant translation infrastructure, including monomorphization support used by other Lean automation projects.[15, 16]

The controller should not indiscriminately send every declaration in the local environment. Select the dependency-relevant facts and instantiate polymorphic schemas only at the types currently required by the proof demand. Record which source theorem and universe/type instantiation each exported formula came from, so reconstruction can apply the actual theorem.

If an abstraction intentionally forgets structure, document its direction: every Lean countermodel should map to an abstract model when an unsatisfiability argument relies on that property. A spurious abstract model then yields no Lean refutation. The final protection remains proof reconstruction in the original context, not an informal assertion that the translation is “basically sound.”

11.3 Candidate generation can be untrusted without being unconstrained

An optional model, learned ranker, or language model can propose a lemma, induction variable, proof sketch, or term. It cannot change the accepted logical rules. It also cannot submit arbitrary Lean commands that introduce new axioms or alter the environment under the guise of a proof term. Restrict the import channel to a typed certificate schema or a term elaborated inside a fixed environment. Resource and size limits protect the checker from pathological proposals.

The design does not require an online model service. The executed prototypes use deterministic enumeration, exact algebra, and local LP proposals. Their algorithms can be useful even in a fully offline implementation.

12 Soundness invariants and adversarial failure modes

12.1 The trust boundary

For the proposed Lean tactic, correctness ultimately rests on the accepted Lean environment and kernel checking of the completed proof. External search code, optimization solvers, heuristics, and speculative graphs need not be trusted to return correct answers. The implemented Python checkers are a research prototype of this separation; their correctness is not established by Lean’s kernel.

An accepted source theorem must have no unresolved expression or universe metavariables, no sorry dependency, and no unexpected new axioms relative to the approved environment. Existing standard axioms and user-approved assumptions are not silently removed or reclassified. A replay artifact should make its dependencies inspectable, for example by normal Lean axiom reporting.

12.2 The invariants to assert in code

Invariant 12.1 (Context validity). *Every proved fact is accompanied by a proof whose free variables and local instances belong to its recorded context, or by an explicit abstraction and instantiation that transports it into the consuming context.*

Invariant 12.2 (Candidate quarantine). *No candidate theorem, model equation, numerical conjecture, or unsolved proof hole is admitted to the proved-fact state. Exploratory assumptions never serve as proof evidence.*

Invariant 12.3 (Induction discipline). *An induction hypothesis is used only with the scope and smaller-argument relationship supplied by its recursor. Generalized parameters may vary; the structural induction index may not be replaced arbitrarily.*

Invariant 12.4 (Certificate binding). *A certificate is checked against the source-derived problem, with the same variable mapping, types, assumptions, and target. A certificate for another polynomial, another box, or a different quantifier scope is not interchangeable.*

Invariant 12.5 (Fail-closed arithmetic). *Numerical tolerances may guide search. Accepted equalities, coefficient signs, partition boundaries, and proof-rule premises are checked exactly.*

12.3 An implementation review checklist

The most dangerous implementation errors are not merely weak heuristics. They are mistakes that accidentally turn search evidence into a stronger assumption. Review at least the following cases explicitly:

Failure mode	Required defense
Self-justifying lemma	Prove candidates in a separate obligation; forbid dependency cycles unless justified by an explicit joint induction principle.
Induction hypothesis applied to the wrong list	Keep the tail/index rigid; let only genuinely generalized binders instantiate.
Branch fact leakage	Record split assumptions and scope; combine branches with a proof, not a global cache insertion.
Floating-point near-equality	Recompute exact rational identities; a small residual is a rejection, not a proof.
Wrong root problem	Bind the certificate to the original reified expression and assumptions, not an oracle-provided replacement.
Missing subdivision region	Derive child boxes from a full split tree and check both children.
Erased dependent type or cast	Preserve typed denotation proofs and explicit transport obligations.
Numerical field proof used for integer division	Use a source-semantics bridge or reject the fragment.
Opaque model element used as a witness	Reify an actual source term with the correct scope; otherwise reject the proposal.
Incomplete external reconstruction	Keep every hole as an unsolved obligation; no success until all are checked.

Failure mode	Required defense
Stale state after metavariable assignment	Invalidate affected caches or restore a consistent snapshot containing the old identities.

Only a subset of these has an executable analogue in the current miniature languages. The tests cover rigid induction tails, unknown/cyclic rewrite rules, malformed paths, sort mismatches, exact arithmetic corruption, wrong constraints, witness corruption, and partition corruption. They do not test a Lean-dependent-type reifier or a real branch-state adapter, because those components have not been implemented.

12.4 Proof size is a separate resource

A solver may find a mathematical argument quickly yet produce a proof too large to elaborate or check efficiently. Track the sizes of certificates, polynomial supports, rewrite traces, and final Lean proof terms separately from search time. Share proof subterms, and prefer a small checked reflection lemma to expanding an enormous algebraic identity into individual rewrites.

A proof DAG can be topologically emitted as local lemmas. Minimize a replay script only by rerunning and rechecking the simplified version. Never delete a dependency because it appears unused in a heuristic trace while remaining inside the actual proof term.

13 A concrete implementation plan

13.1 Module boundaries

A Lean package could use the following organization. These are proposed file names, not claims about files already present in this archive.

```
Forge/Core/GoalSnapshot.lean
Forge/Core/Obligation.lean
Forge/Core/CheckedFact.lean
Forge/Core/Replay.lean
Forge/Search/Controller.lean
Forge/Search/RuleIndex.lean
Forge/Search/Structural.lean
Forge/Induction/CallAnalysis.lean
Forge/Induction/Generalize.lean
Forge/Induction/LemmaFrontier.lean
Forge/Synthesis/AffineWitness.lean
Forge/Synthesis/Recurrence.lean
Forge/Instantiation/ConstrainedMatch.lean
Forge/Instantiation/ModelGuided.lean
Forge/Reify/Polynomial.lean
Forge/Certificate/Farkas.lean
Forge/Certificate/Bernstein.lean
Forge/Backend/Grind.lean
Forge/Backend/SOS.lean
Forge/Backend/SMT.lean
Forge/Tactic.lean
```

The core modules depend only on Lean and a small shared representation. Mathlib-specific reifiers and tactics belong behind separate imports. External processes consume a versioned certificate/problem format and return only untrusted proposals. The package remains useful when those processes are unavailable.

13.2 Milestones with objective exit criteria

First milestone: proof-state controller and baseline parity. Implement exact-state restore, branch-scoped obligations, replay records, and a grind tactic adapter. Its restricted mode must reproduce a pinned baseline suite without introducing new axioms or losing source assumptions.

Second milestone: structural induction and accumulator generalization. Port the miniature list algorithm using real Lean expressions, then extend it to a small set of inductive types. The decisive tests are not just successes: failed generalizations, dependent hypotheses, and wrong-tail induction hypotheses must be rejected or left unsolved.

Third milestone: exact certificates. Implement the affine/Farkas certificate theorem and the recurrence certificate reconstruction. Integrate the existing SOS backend behind the same checked-fact interface. Add Bernstein checking only after the source box and polynomial reifiers have their soundness contracts.

Fourth milestone: missing terms and missing lemmas. Add arithmetic constrained matching, typed lemma-frontier abstraction, and counterexample filtering. Evaluate generalization separately from helper-lemma discovery so the contribution of each is visible.

Fifth milestone: incremental internals and optional external workers. Introduce a version-pinned grind session adapter only where measurements show repeated setup costs dominate. Add SMT and higher-order workers without expanding the trusted base or pretending every external proof rule is already supported.

Sixth milestone: hardening and competitive evaluation. Use holdout benchmarks, per-worker ablations, failure classifications, certificate mutation tests, proof-size measurements, and clean-build replay. A release claiming significant superiority should be based on these results, not on the manufactured successes in this article.

13.3 User interface and explanations

A possible syntax, explicitly *proposed*, is:

```
-- Proposed syntax, not supplied as an executable tactic:
forge [relevant_definitions, useful_theorems]
forge? (induction := true) (witnesses := .affine)
forge (external := false) (maxLemmaDepth := 2)
```

`forge?` should emit a stable proof script or certificate replay plus any local helper lemmas it actually needed. A failure report should distinguish “no induction motive found,” “candidate lemma refuted by an executable example,” “nonlinear certificate search exhausted,” “unsupported source operation,” and “external proof reconstruction incomplete.” It should identify the unsolved obligation rather than merely advise a larger timeout.

The replay should not rerun stochastic discovery or require the original external solver. A proof that only succeeds while a particular model service is available is an unsatisfactory end product when an explicit certificate can be retained instead.

14 Executed experiments and their interpretation

14.1 Environment and reproducibility

The recorded run used Python 3.13.5 on Linux, NumPy 2.3.5, and SciPy 1.17.0. The sparse polynomial representation and all acceptance checks use Python’s standard library, chiefly exact Fraction arithmetic. SciPy is used only for LP proposal generation. SymPy was available but is not needed by the supplied prototypes.

The recorded seeds are 420 for recurrence cases, 421 for witnesses, 422 for square certificates, and 423

for Bernstein cases. The test suite uses separate fixed seeds for its randomized regression checks. The archive includes every accepted certificate and its stated problem, not just aggregate counts.

No Lean executable was available, and an attempted toolchain acquisition did not succeed. Consequently no Lean tactic was run, no imported Lean library combination was compiled, and no kernel-checking time was measured. The file `lean/ReplayExamples.lean` is explicitly labeled uncompiled and is excluded from all success counts.

14.2 Results

Experiment family	Cases	Exact replays	Time (s)
List induction and generalization	6	6	0.005
Polynomial additive recurrences	65	65	0.241
Affine witnesses with Farkas certificates	63	63	0.352
Finite-dictionary square certificates	40	40	0.648
Bernstein box certificates	41	41	0.067
Total certificate cases	215	215	

The times are one local Python run, rounded from `results/results.json`. Each family includes its search and immediate checks; these are not independent hardware-normalized benchmark timings or predictions of Lean performance. The separate replay run verifies all 215 stored certificates without redoing search.

The list family contains four supplied seed statements with searched proofs, the synthesized accumulator specification, and the original specialized specification. Its six proof traces contain respectively 6, 3, 8, 10, 9, and 5 rewrites. These are small proofs, as expected for a deliberately restricted language.

The recurrence family comprises five named power sums and 60 seeded random polynomial increments with rational coefficients and rational initial values. It required 312 candidates in total. The exact step checks generated 247 counterexamples before the 65 final certificates were accepted.

The witness family comprises three named inequality problems, 40 problems whose equalities force affine witnesses, and 20 constructed polyhedral inequality problems. These exercise both the coefficient synthesis and the nonnegative multiplier checks. They are not an evaluation on arbitrary quantified formulas.

The 40 square-certificate problems are *constructed from the same finite dictionary used by the search*. Their 100% success rate therefore tests implementation and rational reconstruction on that designed distribution; it must not be presented as general nonlinear proving coverage.

The 41 Bernstein problems are strictly positive low-degree polynomials made from squares plus positive rational constants. Root-only checking certifies 25. Subdivision certifies all 41, using 82 leaves in total and a maximum split depth of four. A full SOS solver or another tactic may also solve these problems; no comparative claim is made about them.

14.3 Ablations that demonstrate the mechanisms

Problem / mechanism	Restricted attempt	Enabled mechanism
Accumulator reverse	Induction with empty accumulator fixed: no proof.	Changing-argument generalization and typed RHS enumeration: checked proof.
Universal strip $x + 1 \leq y \leq x + 2$	Constant witness template: no certificate.	Affine template: $y = x + 1$, exactly checked.
41 positive box polynomials	Root Bernstein coefficients: 25 certified.	Rational subdivision: 41 certified.
Triangular-number interpolation trap	First four values agree with a false formula.	Exact induction-step identity rejects it.
Diagonal square $(x - y)^2$	Bernstein search at depth four: unknown.	Square certificate is available directly.

These ablations answer the question “does the proposed mechanism do something that its restricted precursor does not?” They do not answer “how many more Mathlib goals does Forge solve than grind?” That second question requires an implemented Lean tactic and a controlled comparison.

14.4 Negative and adversarial tests

All 31 unittest methods passed. Several contain multiple seeded trials, including exact polynomial arithmetic, Bernstein reconstruction, recurrence identities, and semantic cross-checks of the list equations. The suite also rejects a false reverse identity, wrong-tail induction matching, an unknown or self-referential rule, a corrupt rewrite path, and ill-sorted terms.

Arithmetic checks reject negative square weights, certificates attached to a changed target, omitted domain constraints, an incorrect witness coefficient, inexact witness-certificate values, and negative slacks. Partition checks reject missing children, boundary split points, and invalid axes. Strict positivity and weak nonnegativity are tested separately.

The Motzkin polynomial

$$x^4y^2 + x^2y^4 + 1 - 3x^2y^2$$

is a deliberate unsuccessful plain-square-dictionary case. No certificate is reported. The test is a regression boundary for this solver, not evidence that the polynomial is negative. Similarly, insufficient recurrence degree or insufficient grammar size returns no proof, and a depth-limited Bernstein failure remains unknown.

14.5 What the evidence does and does not establish

It establishes that the included programs ran, that the reported certificates passed their specified exact Python checks, and that the listed negative cases were rejected. Together with the mathematical arguments, it supports the feasibility of several workers and the usefulness of generalization, certificate synthesis, and subdivision in their stated fragments.

It does not establish correctness of a Lean implementation that has not been written, completeness outside those fragments, robustness of a dependent-type reifier, improved performance on a representative corpus, or a measured advantage over current grind. There is also shared substrate between search and replay: both use the same term or polynomial representation. “Search-free replay” is not the same as an independently implemented formal verification of that representation.

15 How to demonstrate genuine superiority over grind

15.1 Freeze the comparison, including its theorem knowledge

Use a pinned Lean release, a compatible pinned Mathlib revision, fixed imported modules, and recorded backend versions. Preserve each source goal and its original hypotheses. Avoid accidentally making the tested theorem itself, a renamed duplicate, or a future theorem dependent on it available through library retrieval. This is especially important when benchmarks are extracted from an already compiled mathematical library.

Compare ordinary grind, grind with explicit relevant equations/lemmas, and the current shared-state bit-vector route where applicable. Also compare Aesop, a simple tactic portfolio, and appropriate specialized tools such as SOS, Duper, or Lean-SMT. A system that wins only against an artificially weak baseline has not demonstrated the desired improvement.

For structural problems, distinguish three input conditions: definitions only; a common relevant helper-lemma set; and an oracle-assisted set containing the handwritten proof's helper lemmas. The gap between the first two conditions measures lemma acquisition, while the last helps separate discovery failures from local proof failures.

15.2 The benchmark families

The most informative families are recursive data-structure theorems needing induction or changing accumulators; polynomial cost and invariant proofs; quantified statements needing new arithmetic terms; constructive existential specifications; mixed guarded arithmetic and rewriting; and higher-order or indexed-datatype obligations.

Add a large control set of goals already handled well by grind, including its arithmetic and bit-vector strengths. The control set detects whether the new controller wastes resources on familiar cases. DT-TBench, introduced with Canonical-min, is relevant additional material for the typed-inhabitation worker, but does not replace a Lean integration benchmark.[19]

Hold out whole problem families when tuning heuristic weights. Renaming variables or slightly changing constants in a training problem does not create an independent test set for lemma-discovery heuristics.

15.3 Metrics and ablations

Report solved fraction, wall time, Lean heartbeats where meaningful, peak memory, proof/certificate size, reconstruction time, and kernel checking time. Record failed translations, unsupported fragments, timeouts, rejected certificates, and residual reconstruction goals separately. A solver verdict with an unclosed proof hole is not a solved benchmark.

Run at least the following ablations: no induction; induction without generalization; no helper-lemma discovery; no constrained instantiation; constant-only witnesses; no nonlinear certificate worker; and stateless local solver restarts versus the incremental adapter. Include the simple portfolio with the same total budget to measure the controller's interactions rather than merely its larger collection of tools.

The principal evidence for "significantly more capable" should be a substantial held-out solved-set increase in structural and synthesis-heavy families, while retaining most control-set successes and producing acceptable proof-checking costs. A speed claim requires repeated controlled runs; a capability claim requires concrete additional checked goals. Neither follows from the present prototype counts alone.

16 Assessment and recommended starting point

The most promising first implementation is *not* a complete new SMT solver and not a rewrite of grind. It is a narrow Lean controller that combines automatic generalization and induction with an exact witness and

recurrence-certificate pipeline. This already changes the shapes of proofs that can be found automatically. It also provides clear interfaces for adding stronger nonlinear and quantified reasoning afterward.

The highest-risk engineering tasks are preserving dependent contexts through generalization, maintaining valid state across local solver restarts or snapshots, producing compact proof reconstruction, and keeping candidate lemmas out of the proved state. The numerical search itself is comparatively easy to sandbox because an exact certificate can reject a bad proposal.

The Python experiments give a concrete foothold: a synthesized accumulator lemma, exact recurrence invariants, parametric witnesses with proof multipliers, and complementary polynomial certificates. The proposed Lean architecture explains how these become cooperating proof workers rather than isolated demos. That is a technically plausible route to a tactic substantially more useful than unassisted local saturation on the targeted goal families. Its eventual superiority should be earned by the pinned, proof-checked benchmark program just described.

A Artifact inventory and commands

The archive is organized as follows:

```
article/
  forge.tex, 01-architecture.tex, 02-algorithms.tex
  03-integration.tex, 04-evaluation.tex, references.tex
  forge.pdf
prototype/
  forge_proto/{poly,linear,induction,recurrence,
               witness,positivity}.py
  tests/test_forge.py
  run_experiments.py
  replay_certificates.py
  requirements.txt, README.md
results/
  results.json, experiments.log, tests.log, replay.log
  certificates/{induction,recurrences,witnesses,
               sos,bernstein}.json
lean/
  ReplayExamples.lean
README.md, SOURCES.md
```

From prototype/, reproduce the executed work with:

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_experiments.py --output ../results
python -S replay_certificates.py ../results/certificates
```

The -S option disables third-party site-package loading. The last command succeeds because checking does not require the numerical proposer. It reports 6 induction, 65 recurrence, 63 witness, 40 square, and 41 Bernstein certificates, totaling 215.

Build this article from article/ with two runs of LuaLaTeX:

```
lualatex -interaction=nonstopmode -halt-on-error forge.tex
lualatex -interaction=nonstopmode -halt-on-error forge.tex
```

The document uses Linux Libertine O, Lato, and DejaVu Sans Mono installed as system fonts. Font files are not included. Another available font family can be selected by changing the three fontspec declarations.

B Certificate representations

B.1 Polynomials

A polynomial stores a positive variable count and a sorted list of nonzero rational coefficients paired with nonnegative exponent tuples. Rational coefficients are serialized as exact strings such as "3/7". Canonical construction rejects malformed exponent vectors, zero entries, and duplicate or unsorted serialized terms.

The variable table binding these indices to Lean expressions is a separate production concern. The Python arithmetic certificates use fixed mathematical variables and do not implement that Lean source map.

B.2 Induction traces

An induction certificate contains an equation, its induction variable, and two equality traces. Each equality trace has a sequence of rewrites for its left and right sides. A rewrite is a pair (rule_name, subterm_path). The checker derives the base/step contexts and the only permitted induction hypothesis; these are not arbitrary claims supplied by the certificate.

The fixture stores certificates in dependency order. Replay installs each one only after checking it against defining equations and earlier installed lemmas. The experiment's seed statements therefore do not enter as axioms.

B.3 Affine and square certificates

An affine witness certificate stores one affine expression per output, one nonnegative multiplier row per target inequality, and one nonnegative slack per target inequality. Verification is exact coefficient equality in (4)–(5).

A square certificate stores weighted square roots with an optional index of a source nonnegative constraint. It is checked against the separately supplied target and constraint list. The current format has no arbitrary equality multipliers; adding those is a straightforward extension of the identity checker, but not an implemented feature claimed by the experiments.

B.4 Bernstein trees

A leaf contains only its kind. A split contains an axis, exact split point, and two subtrees. The checker derives all bounds from the root domain and recomputes all leaf coefficients. Node/depth limits are operational rejection limits. They are not mathematical assumptions about positivity.

B.5 A production interchange format

A production format should additionally carry a schema version, source-goal identifier, environment/encoding revision, variable-map identifier, required assumptions, and claimed result kind. The Lean process resolves these identifiers against its own state rather than accepting oracle-provided expressions as authoritative. Resource limits should bound integer bit sizes, monomial counts, degrees, tree nodes, and proof depth before expensive checking.

C Known limitations of the delivered code

The prototypes do not implement the full obligation scheduler, actual Lean snapshots, native grind state access, dependent typing, general quantifier instantiation, higher-order synthesis, or external SMT proof reconstruction. The square search is a finite linear dictionary, not semidefinite programming. Affine synthesis is over rational/real non-strict polyhedral constraints, not mixed-integer or arbitrary quantified arithmetic. The recurrence solver handles additive polynomial increments only.

The exact checkers are ordinary Python programs. They have executable regression tests and elemen-

tary soundness arguments for their formats, but are not themselves formally verified. Several share representation and matching routines with their search counterparts. This is appropriate for a prototype and is precisely why the Lean implementation must reconstruct proofs or use a checker with a proved soundness theorem.

The Lean examples are uncompiled illustrations with no sorry placeholders. Absence of sorry is not evidence that they compile, and no such claim is made. The package is an actionable research/design artifact with tested algorithmic components, not a released drop-in replacement for grind.

References

- [1] Lean project. *The grind tactic*. The Lean Language Reference. Accessed September 14, 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Lean project. *Lean 4.34.0 (2026-09-14)*. Release notes, especially “Bit-blasting Inside grind =>.” <https://lean-lang.org/doc/reference/latest/releases/v4.34.0/>.
- [3] Lean project. *Linear Integer Arithmetic*. The Lean Language Reference. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Integer-Arithmetic/>.
- [4] Lean project. *E-matching*. The Lean Language Reference. https://lean-lang.org/doc/reference/latest/The--grind--tactic/E__matching/.
- [5] Lean project. *Grind/Main.lean*, tag v4.34.0. Inspected through the GitHub repository interface. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Main.lean>.
- [6] Lean community. *Aesop*. Project documentation, including search-tree and rule descriptions. Accessed September 14, 2026. <https://github.com/leanprover-community/aesop>.
- [7] Cole Kurashige, Ruyi Ji, Aditya Giridharan, Mark Barbone, Daniel Noor, Shachar Itzhaky, Ranjit Jhala, and Nadia Polikarpova. *CCLemma: E-Graph Guided Lemma Discovery for Inductive Equational Proofs*. Proceedings of the ACM on Programming Languages 8 (ICFP), article 264, 2024. DOI: 10.1145/3674653. Author/conference abstract and artifact description: <https://icfp24.sigplan.org/details/icfp-2024-papers/29/CCLemma-E-Graph-Guided-Lemma-Discovery-for-Inductive-Equational-Proofs>.
- [8] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. *egg: Fast and Extensible Equality Saturation*. Proceedings of the ACM on Programming Languages 5 (POPL), 2021. <https://arxiv.org/abs/2004.03082>.
- [9] Marcus Rossel. *lean-egg*. Project README, including discontinuation notice. Accessed September 14, 2026. <https://github.com/marcusrossel/lean-egg>.
- [10] Mathlib contributors. *Mathlib.Tactic.Linarith.Frontend*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [11] Mathlib contributors. *Mathlib.Tactic.Positivity.Basic*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Positivity/Basic.html.
- [12] Mathlib contributors. *Mathlib.Tactic.LinearCombination*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html.
- [13] Lean sos contributors. *SOS: nonlinear real arithmetic in Lean 4*. Project README, architecture, certificate interface, and limitations. Accessed September 14, 2026. <https://github.com/leanprover/sos/blob/main/README.md>.
- [14] Lean community. *Duper*. Project documentation. <https://github.com/leanprover-community/duper>.
- [15] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. *Lean-auto: An Interface Between Lean 4 and Automated Theorem Provers*. 2025. <https://arxiv.org/abs/2505.14929>. Project: <https://github.com/leanprover-community/lean-auto>.

- [16] Lean-SMT contributors. *lean-smt*. Project README, supported theories, monomorphization, and proof-reconstruction behavior. <https://github.com/ufmg-smite/lean-smt>. See also the associated 2025 paper: <https://arxiv.org/abs/2505.15796>.
- [17] Z3 project. *Quantifiers*. Z3 Guide, including patterns and model-based quantifier instantiation. <https://microsoft.github.io/z3guide/docs/logic/Quantifiers/>.
- [18] Chase Norman and Jeremy Avigad. *Canonical for Automated Theorem Proving in Lean*. 2025. <https://arxiv.org/abs/2504.06239>.
- [19] Chase Norman and Jeremy Avigad. *Implementing Dependent Type Theory Inhabitation and Unification*. 2026. <https://arxiv.org/abs/2603.01463>.
- [20] SciPy contributors. *scipy.optimize.linprog*. Official API documentation. The live documentation was inspected; the executed environment used SciPy 1.17.0. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>.
- [21] MOSEK contributors. *MOSEK Modeling Cookbook*, linear optimization, infeasibility certificates, and duality. Official mathematical documentation. <https://docs.mosek.com/modeling-cookbook/linear.html>.